

Criterion: Backend Architecture & API Delivery

Architecture Decision Record

Status

Status: Accepted

Date: 2026-01-05

Context

The platform needs a simple, reliable REST API for a read-heavy disease library (pagination, filtering, search) with bilingual content (EN/RU). The backend must be maintainable for a single-developer project and easy to run locally in Docker.

Decision

Implement a TypeScript backend on Node.js with Express using a layered structure (controllers ? services ? repositories), Prisma for DB access, optional Redis caching, centralized error handling, and OpenAPI docs generated from JSDoc.

Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Fastify	High performance; schema-first	Refactor cost	Express already fits the project tool...
NestJS	Strong structure; DI patterns	More framework overhead	MVP benefits from lighter architecture
FastAPI (Python)	Great for APIs; fits analytics language	Two-stack FE/BE split	Backend already implemented in TS and...

Consequences

Positive: predictable request lifecycle, documented endpoints, reproducible local stack.

Negative: DI (Inversify) is not consistently applied across all modules.

Neutral: Redis caching can be expanded only for hot paths.

Implementation Details

Key Implementation Decisions

- * Middleware pipeline: JSON ? CORS ? routes ? notFound ? error middleware.
- * Explicit domain modules (disease, source) with services and repositories.

- * OpenAPI generated close to route/controller definitions.
- * Redis cache as an optimization layer (safe fallback to DB on miss).

Project Structure

```
backend/src/
+-- app/                # Express app composition
+-- routes/             # Route mounting (/health, /diseases, /sources)
+-- disease/            # Disease domain module (controller/service/repo/cache)
+-- source/             # Source domain module
+-- config/             # env config keys, swagger, Sentry instrumentation
+-- middlewares/        # notFound + error middleware
+-- errors/             # ApiError + createApiError factory
+-- cache/              # Redis client
+-- utils/              # Winston logger + Morgan request logger

backend/prisma/
+-- schema.prisma
+-- migrations/
+-- seed.ts
```

Key Implementation Decisions

Decision	Rationale
Express middleware pipeline (json, co...	Standard, predictable request lifecycle
Controller/service/repository split	Keeps business logic testable and sep...
OpenAPI via JSDoc (express-jsdoc-swag...	Co-locates endpoint docs with impleme...
Redis client with env-configured host...	Enables docker-compose and local dev ...
Graceful shutdown handlers	Avoids abrupt disconnects and resourc...

Code Examples

Routes and module mounting:

```
// backend/src/routes/index.ts
router.use('/health', healthController.router);
router.use('/diseases', diseaseController.router);
router.use('/sources', sourceController.router);
```

Error handling via typed API errors:

```
// backend/src/disease/disease.controller.ts
if (isNaN(skipValue) || skipValue < 0) {
  throw createApiError.badRequest(msg.SKIP_PARAM_INCORRECT);
}
```

Diagrams

- * Container/service wiring is described in docs/02-technical/deployment.md.

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Express application setup with router	[OK]	backend/src/app/app.ts
2	REST endpoints for diseases/symptoms + ...	[OK]	backend/src/routes/index.ts, backend/...
3	Pagination/filtering/search for diseases	[OK]	backend/src/disease/disease.controller...
4	Locale-aware responses (en/ru)	[OK]	locale query handling in disease cont...
5	OpenAPI documentation available	[OK]	JSDoc annotations + backend/src/confi...
6	Centralized error middleware	[OK]	backend/src/middlewares/errorMiddlewa...
7	Redis cache client wired	[OK]	backend/src/cache/redisClient.ts
8	Dependency Injection consistently used	[OK]	Inversify container exists (backend/s...

Known Limitations

Limitation	Impact	Potential Solution
DI is not consistently applied	Harder to swap implementations for te...	Move instantiation into container bin...
Some ?no results? cases return 200 + ...	Clients must handle two shapes (array...	Consider returning empty arrays with ...

References

- * backend/package.json
- * backend/src/app/app.ts
- * backend/src/routes/index.ts
- * backend/src/disease/disease.controller.ts
- * backend/src/cache/redisClient.ts
- * backend/src/config/instrument.js